BASIC/CREXX

A BASIC interpreter written in CREXX

Language Reference and User's Guide

Release 1 • Build 1
for CREXX 1.0.0 beta 3 and later

Publication BCX-0001-0
File No. BASIC-01

First Edition (October 2026)

This edition applies to BASIC/CREXX Release 1 (build 1, 9 October 2026), an interpreter for the BASIC language written in CREXX Level B, and to all subsequent builds until otherwise indicated in new editions.

BASIC/CREXX is a hobbyist project. It is not a product of, and this manual is not a publication of, Microsoft Corporation or the FreeBASIC project. Microsoft, GW-BASIC, QuickBASIC and FreeBASIC are names of products and languages, used here only to identify the dialects that this software accepts and from which it descends.

CREXX is an open-source REXX implementation and is maintained by its own community. BASIC/CREXX needs it to run.

The project repository holds the interpreter source, the examples and the regression tests from which every statement in this manual can be checked. Comments on this publication may be addressed to the project.

Contents

Preface ... 1

Chapter 1. Introduction ... 2
- How BASIC/CREXX is built ... 2
- Status ... 2

Chapter 2. Installing and Running ... 3
- What you need ... 3
- Installation steps ... 3
- Running a program ... 3
- The C version ... 3
- Command-line options ... 4
- Interactive mode ... 4
- Exit status ... 5
- Testing ... 5

Chapter 3. The BASIC Language ... 6
- Program layout ... 6
- Names and data types ... 6
- Constants ... 6
- Expressions and operators ... 6
- Arrays ... 7

Chapter 4. Statements ... 8
- Assignment and declaration ... 8
- Decisions and loops ... 8
- Data ... 9
- Program control and environment ... 9

Chapter 5. Procedures and Functions ... 11
- SUB and FUNCTION ... 11
- Parameters ... 11
- Scope of variables ... 11
- DEF FN ... 11
- GOSUB ... 12
- Names that duplicate built-in functions ... 12

Chapter 6. Input and Output ... 13
- PRINT ... 13
- PRINT USING ... 13
- INPUT and LINE INPUT ... 14
- Files ... 14
- Reading the keyboard directly ... 15

Chapter 7. Built-in Functions ... 16
- Numeric functions ... 16
- String functions ... 16
- System and file functions ... 17

Chapter 8. Errors and Error Handling ... 18
- Runtime errors ... 18
- ON ERROR ... 18

Chapter 9. How BASIC/CREXX Works ... 19
- Tokens ... 19
- The link pass ... 19
- Execution ... 19
- Performance ... 19
- Building ... 19

Chapter 10. Restrictions and Known Problems ... 20

Appendix A. Command Summary ... 21
- Command line ... 21
- Environment ... 21
- Return codes ... 21

Appendix B. Contents of the Repository ... 22

Appendix C. Differences between the Dialects ... 23

Appendix D. Sample Programs ... 24

Preface

This manual introduces BASIC/CREXX, a BASIC interpreter written in CREXX, tells you how to install and run it, and describes the BASIC language it implements: the statements, operators and functions of Microsoft BASIC (GW-BASIC and QuickBASIC) together with a large part of the FreeBASIC dialect.

It is written for the programmer who already knows BASIC, or who wants to run existing BASIC programs on a system that has CREXX. Where BASIC/CREXX behaves as the Microsoft or FreeBASIC implementations do, this manual says little; where it differs, it says how.

How this manual is organized

- **Chapter 1, Introduction**, says what BASIC/CREXX is, how it is built and what it supports.
- **Chapter 2, Installing and Running**, covers the launcher, the command-line options, interactive mode and the regression tests.
- **Chapter 3, The BASIC Language**, describes program layout, names, data types, expressions and arrays.
- **Chapter 4, Statements**, is the reference for every statement, grouped by purpose.
- **Chapter 5, Procedures and Functions**, covers SUB, FUNCTION, DEF FN and GOSUB.
- **Chapter 6, Input and Output**, describes PRINT, PRINT USING, INPUT and file handling.
- **Chapter 7, Built-in Functions**, is the reference for the intrinsic functions.
- **Chapter 8, Errors and Error Handling**, lists the error codes and explains ON ERROR.
- **Chapter 9, How BASIC/CREXX Works**, describes the tokenizer, the link pass and the executor for those who wish to extend the interpreter.
- **Chapter 10, Restrictions and Known Problems**, lists what does not work yet.
- **Appendix A** summarizes the command line, **Appendix B** lists the files of the repository, **Appendix C** compares the three dialects, **Appendix D** lists the sample programs.

Conventions

Statements and program text are shown in monospaced type. In syntax descriptions, uppercase words are keywords and are typed as shown; lowercase words stand for values you supply; items in square brackets [ ] are optional; a vertical bar | separates alternatives; and an ellipsis ... means that the preceding item may be repeated. BASIC keywords are not case sensitive: PRINT, Print and print are the same statement.

Examples of programs and terminal sessions are shown in framed figures. In a terminal session the output of the interpreter follows the command that produced it.

MS style means the number layout of GW-BASIC and QuickBASIC; *FB style* means that of FreeBASIC. Both are selectable (see Chapter 2).

Chapter 1. Introduction

BASIC/CREXX is a complete BASIC interpreter written in CREXX, the open-source REXX implementation. It runs classic Microsoft BASIC programs, with or without line numbers, and a large part of the FreeBASIC dialect. The interpreter is a single CREXX source file of about 5000 lines, basic.crexx, written in CREXX Level B, the statically typed level of the language.

BASIC/CREXX gives its users:

- **Both classic and structured BASIC.** Numbered lines with GOTO and GOSUB, and structured programs with SUB, FUNCTION, SELECT CASE, DO...LOOP and block IF, in the same program if you wish.
- **Procedures.** SUB and FUNCTION with recursion, BYREF and BYVAL parameters, array parameters, STATIC and SHARED variables, and the old DEF FN single-line functions.
- **Formatted output.** PRINT with zones and TAB, and PRINT USING with numeric, string, currency, sign and exponent fields.
- **Files.** Sequential text files opened for input, output or append, with PRINT #, WRITE #, INPUT # and LINE INPUT #.
- **Error handling.** ON ERROR GOTO, RESUME, ERR and ERL, with the standard Microsoft error numbers.
- **An interactive prompt.** Enter numbered lines, LIST, RUN and LOAD programs, or type a statement and see it executed at once.

How BASIC/CREXX is built

A program is read once and converted to a stream of tokens held in parallel arrays. A link pass then resolves what is structural: it matches IF with ELSE and END IF, FOR with NEXT, DO with LOOP, SELECT CASE with its CASE clauses, SUB and FUNCTION with their ends, DEF FN definitions, labels, and the items of DATA statements. The executor then walks the token array, evaluating expressions with a precedence-climbing evaluator. No program text is scanned while the program runs.

The interpreter is compiled by the CREXX compiler to a bytecode file, which the CREXX virtual machine runs. The first run of the launcher performs this compilation; later runs start in under a second.

Status

The figure below shows the major areas of the language and their status in this release.

Area	Content	Status
Core	Variables, expressions, PRINT, IF, FOR, WHILE, DO, GOTO, GOSUB, ON...GOTO	complete
Data	DIM, REDIM, ERASE, CONST, DATA, READ, RESTORE, DEFINT and the other DEFxxx	complete
Procedures	SUB, FUNCTION, DEF FN, CALL, recursion, BYREF, STATIC, SHARED	complete
Structure	SELECT CASE, EXIT, CONTINUE, compound assignment	complete
Output	PRINT, PRINT USING, TAB, SPC, WRITE, CLS, LOCATE, COLOR	complete (screen control with -ansi)
Input	INPUT, LINE INPUT, INPUT$, INKEY$	complete
Files	Sequential files, EOF, FREEFILE	complete
Errors	ON ERROR GOTO, RESUME, ERROR, ERR, ERL	complete
User types	TYPE ... END TYPE, UNION, ENUM	planned
Random files	PUT, GET, SEEK, LOF, FIELD	planned
Graphics, sound	SCREEN, PSET, LINE, CIRCLE, SOUND, PLAY	not planned

Figure 1. BASIC/CREXX language areas

Chapter 2. Installing and Running

What you need

BASIC/CREXX needs only the CREXX toolchain: the crexx driver together with the compiler (rxc), assembler (rxas) and virtual machine (rxvm) that it calls. CREXX 1.0.0 beta 3 was used to build and test the interpreter. The driver can be installed from a CREXX release package, or built from source with CMake; see the CREXX project for instructions.

The launcher script basic is a POSIX shell script. On systems without a shell, run the interpreter with the crexx driver directly, as shown below.

Installation steps

1. Install CREXX and make sure the crexx command is on your PATH. If it is not, set the environment variable CREXX to the full path of the driver.
2. Get the BASIC/CREXX repository (git clone https://github.com/mikebeer/basic).
3. Run any program with the launcher: ./basic examples/hello.bas. The first run compiles the interpreter to bytecode and takes about half a minute. The compiled file, basic.rxbin, is kept next to basic.crexx.
4. Run the regression tests with ./tests/run_tests.sh. All of them should report ok.

Running a program

The figure below shows a program run from the launcher.

```
10 REM Classic MS BASIC: Hello World with a loop
20 PRINT "Hello from BASIC on CREXX!"
30 FOR I = 1 TO 5
40   PRINT I, I * I, SQR(I)
50 NEXT I
60 END

$ ./basic examples/hello.bas
Hello from BASIC on CREXX!
 1             1             1
 2             4             1.4142135623731
 3             9             1.73205080756888
 4             16            2
 5             25            2.23606797749979
```

Figure 2. A classic program, and its output

Without the launcher, give the driver the interpreter source and pass the program after -args:

```
crexx basic.crexx -args examples/hello.bas
crexx basic.crexx -nocompile -args examples/hello.bas
```

Figure 3. Running without the launcher

The first form compiles basic.crexx each time (slow); the second reuses an existing basic.rxbin. Everything after the program name is passed to the BASIC program and can be read with COMMAND$.

The C version

The directory c holds a second implementation of the interpreter, a port to strict ISO C89 that needs no CREXX at all. It reads the same programs and prints the same output; the regression tests in the directory tests are shared by both versions. It is meant to run on small systems such as VM/370 under CMS, where a C compiler is available but CREXX is not.

```
cd c
make                      # builds ./basic from the single file basic_all.c
make test                 # runs ../tests against ./basic
make strict               # gcc and clang -std=c89 -pedantic, sanitizers
make deck                 # deck/basic_deck.c, lines of at most 71 columns
./basic ../examples/hello.bas
./basic -fb prog.bas       # the options are the same as for the CREXX version
```

Figure 4. Building and running the C version

The C version formats numbers exactly as the CREXX version does: a number is printed with up to 15 significant digits, whole numbers below 10 to the power 15 are printed in full, and the exponent form is used only for numbers of 10 to the power 18 or more and for numbers below 0.000001. The functions that work on bit patterns (AND, OR, XOR, NOT, EQV, IMP, SHL, SHR, HEX$, OCT$, BIN$) use 64-bit two-complement arithmetic that is built from 32-bit halves, so the results do not depend on the size of a long integer. Strings are strings of bytes: where the CREXX version counts UTF-8 characters (LEN, MID$, ASC and CHR$ for codes above 127), the C version counts bytes. The benchmark programs in the directory bench run under both versions and print the same results; bench/run_bench.sh times them.

Command-line options

Options precede the program name. The figure below lists them.

Option	Effect
-ms or -qb	MS style output (the default). A number prints with a leading space for its sign and a trailing space: PRINT 5 shows " 5 ".
-fb	FreeBASIC style output. A number prints with a leading space only: PRINT 5 shows " 5". STR$(5) has no leading space.
-ansi	CLS, LOCATE and COLOR send ANSI escape sequences to the terminal. Without this option they do nothing, so output redirected to a file stays clean.
-dump	List the token stream and the variable table instead of running the program. Used to debug the interpreter.
-h	Display a summary of the options and exit.

Figure 5. Command-line options

Interactive mode

Started without a program name, the interpreter prompts with Ok. A line that begins with a number is stored in the program, replacing any line with the same number; a number alone deletes the line. Any other line is executed at once as an immediate statement, and variables keep their values between immediate statements and after RUN. The figure below lists the commands.

Command	Effect
RUN	Clear all variables and run the stored program.
LIST	List the stored program.
NEW	Delete the stored program.
LOAD "file"	Replace the stored program by the contents of a file. Lines of an unnumbered program are numbered 10, 20, 30...
SYSTEM	Leave the interpreter (QUIT and EXIT do the same).

Figure 6. Interactive commands

```
$ ./basic
CREXX BASIC.  Commands: RUN LIST NEW LOAD "file" SYSTEM
Ok
10 FOR I=1 TO 3
20 PRINT I
30 NEXT
Ok
RUN
 1
 2
 3
Ok
PRINT 2+3
 5
Ok
SYSTEM
```

Figure 7. An interactive session

Exit status

The interpreter ends with return code 0 when the program ends normally, and 1 when a runtime error was not handled by the program or when the program could not be loaded or linked. An unhandled error is reported on the standard error stream in the form "Division by zero in 20", where the number is the BASIC line number or, for programs without line numbers, "in line n" with the source line.

Testing

The directory tests holds regression programs. Each NAME.bas has an expected-output file NAME.out and optionally NAME.in (standard input) and NAME.flags (interpreter options). tests/run_tests.sh runs them all and prints ok or FAIL for each; with the argument --update it rewrites the expected files, which should be done only after a change has been checked by hand.

Chapter 3. The BASIC Language

Program layout

A program is a text file with one or more statements per line. Statements on one line are separated by a colon. A line may begin with a line number (an integer) or a label (a name followed by a colon); either can be the target of GOTO, GOSUB, RESTORE and ON ERROR. A program need not use line numbers at all, and line numbers need not be in order, because the interpreter finds the target by value, not by position.

Comments begin with REM, which takes the rest of the line, or with an apostrophe, which may follow a statement. A line that begins with # is ignored.

```
Print "counting"
i = 0
again:
i += 1
Print i;
If i < 5 Then GoTo again
Print

counting
 1  2  3  4  5
```

Figure 8. Program with labels instead of line numbers

Names and data types

A name begins with a letter and continues with letters, digits, the underscore and the period. Names are not case sensitive. A type character at the end of a name fixes the type of the variable; a name without one has the type given by a DEFxxx statement, or else is numeric. The figure below lists the types.

Suffix or declaration	Type	Notes
$ or AS STRING	string	Any length. Character codes 0 to 255 and beyond.
% & or AS INTEGER, LONG, SHORT, BYTE, BOOLEAN	integer	Values are rounded to a whole number when assigned.
! # or AS SINGLE, DOUBLE	floating point	Held as 64-bit floating point; SINGLE is not rounded to 7 digits.
(no suffix)	floating point	Unless changed by DEFINT, DEFSTR and the like.

Figure 9. Variable types

A numeric value that is not whole is held as a 64-bit binary floating point number, and an integer is held in the same form, so integers are exact up to 2 to the power 53. A string variable that has not been assigned is the empty string, and a numeric variable is zero. The names of different types are different variables: A, A$ and A% are three variables.

Constants

A numeric constant is written in decimal, with an optional fraction and an exponent introduced by E or D (1.5E-3, 2D10). Hexadecimal, octal and binary constants are written &H1F, &O17 and &B1011. A string constant is enclosed in double quotes; a double quote inside is written twice. CONST gives a name to a constant value.

Expressions and operators

Expressions are evaluated according to the precedence in the figure below, highest first. Operators on one row have equal precedence and are evaluated left to right.

Operators	Meaning
^	exponentiation
- (unary), +	negation

Operators	Meaning
* /	multiplication, division
\	integer division
MOD	remainder of integer division
SHL SHR	shift left, shift right
+ -	addition and concatenation, subtraction
&	string concatenation (numbers are converted to text)
= <> < > <= >=	comparison; true is -1, false is 0
NOT	bitwise complement
AND	bitwise and
OR	bitwise or
XOR	bitwise exclusive or
EQV	bitwise equivalence
IMP	bitwise implication

Figure 10. Operator precedence

Comparison gives -1 for true and 0 for false, so that NOT, AND and OR work as both logical and bitwise operators. Strings are compared by character code, so "B" is less than "a". Comparing a string with a number is a Type mismatch error. The operators AND, OR, XOR, EQV, IMP and NOT, and the shifts, work on whole numbers; any fraction is rounded away first.

The compound assignments +=, -=, *=, /=, \=, ^= and &= (FreeBASIC) are statements, not operators: A += 1 is the same as A = A + 1.

Arrays

An array is created by DIM, or automatically the first time it is used, with subscripts 0 to 10 in each dimension. Up to three dimensions are supported. Subscripts begin at 0 unless OPTION BASE 1 has been given, or the declaration states bounds with TO. The figure below gives examples.

```
DIM a(10)              ' 11 elements, 0 to 10
DIM b(1 TO 5) AS STRING
DIM g(2, 3) AS DOUBLE  ' 3 by 4 elements
REDIM PRESERVE v(1 TO 20)
OPTION BASE 1
```

Figure 11. Array declarations

Using a subscript outside the bounds is error 9, Subscript out of range. UBOUND(a, n) and LBOUND(a, n) return the bounds of dimension n. REDIM changes the bounds of an array; with PRESERVE the old values of a one-dimensional array are kept. ERASE removes an array.

Chapter 4. Statements

This chapter describes every statement. In the syntax column, a statement with the same name in the two dialects is described once. Statements that fill several lines of a program (block IF, SELECT CASE, DO, SUB) are shown with their closing statement.

Assignment and declaration

Statement	Syntax	Description
LET	`[LET] var = expr`	Assigns a value. LET is optional. The variable may be an array element.
+= -= *= /= \= ^= &=	var op= expr	Compound assignment: var = var op expr.
DIM	`DIM [SHARED] name[(bounds)] [AS type] [= expr], ...` `DIM [SHARED] AS type name, name...`	Declares variables and arrays. bounds is n or lo TO hi, up to three. DIM SHARED makes a module variable visible in procedures. A scalar may have an initial value.
REDIM	`REDIM [PRESERVE] name(bounds) [AS type]`	Changes the bounds of an array.
ERASE	`ERASE name, ...`	Deletes arrays.
CONST	`CONST name = expr [, name = expr ...]`	Defines named constants. A constant defined outside a procedure is visible everywhere.
STATIC	`STATIC name[(bounds)] [AS type]`	In a procedure, declares a variable that keeps its value between calls.
DEFINT, DEFLNG, DEFSNG, DEFDBL, DEFSTR	`DEFINT letter[-letter], ...`	Sets the default type of names that begin with the given letters.
OPTION BASE	`OPTION BASE 0|1`	Sets the lowest subscript of arrays declared without explicit bounds.
SWAP	`SWAP var1, var2`	Exchanges the values of two variables of the same type.
CLEAR	`CLEAR`	Sets all variables to zero or empty, removes arrays and restores the DATA pointer.

Figure 12. Assignment and declaration statements

Decisions and loops

Statement	Syntax	Description
IF (single line)	`IF cond THEN stmts [ELSE stmts]` `IF cond GOTO label`	If the condition is not zero the statements after THEN run. THEN may be followed by a line number or label, meaning GOTO. IF statements may be nested on one line; an ELSE belongs to the nearest IF.

IF (block)	`IF cond THEN`
	`  stmts`
	`[ELSEIF cond THEN`
	`  stmts]...`
	`[ELSE`
	`  stmts]`
	`END IF`
		THEN must be the last item on its line. ELSEIF may be used any number of times.

SELECT CASE	`SELECT CASE expr`
	`  CASE list`
	`  stmts`
	`  CASE ELSE`
	`  stmts`
	`END SELECT`
		Runs the first CASE whose list contains the value. A list holds values separated by commas, ranges a TO b, and tests IS op value. Strings and numbers may be used, not mixed.

FOR	`FOR var = start TO end [STEP n]`
	`  stmts`
	`NEXT [var [, var...]]`
		Counts. The body does not run if the start is already beyond the end. NEXT may name several variables to close several loops.

WHILE	`WHILE cond`
	`  stmts`
	`WEND`
		Repeats while the condition is not zero.

DO	`DO [WHILE|UNTIL cond]`
	`  stmts`
	`LOOP [WHILE|UNTIL cond]`
		Repeats. The condition may be at the top, at the bottom or absent.

Statement	Syntax	Description
EXIT	`EXIT FOR|DO|WHILE|SELECT|SUB|FUNCTION`	Leaves the innermost block of the kind named, or the procedure.
CONTINUE	`CONTINUE FOR|DO|WHILE`	Goes to the next iteration of the innermost loop of that kind.
GOTO	`GOTO label|line`	Transfers control.
GOSUB, RETURN	`GOSUB label|line` `RETURN`	Calls a routine within the program; RETURN goes back to the statement after the GOSUB.
ON ... GOTO	`ON expr GOTO l1, l2, ...` `ON expr GOSUB l1, l2, ...`	Jumps to the nth label. If expr is 0 or greater than the number of labels, the next statement runs.
END, STOP, SYSTEM	`END`	Ends the program. All files are closed.

Figure 13. Control statements

```
Dim sieve(2 To 100) As Integer
For i = 2 To 100: sieve(i) = 1: Next
For i = 2 To 100
  If sieve(i) = 1 Then
    For j = i * i To 100 Step i
      sieve(j) = 0
    Next j
  End If
Next i

Select Case score
  Case Is >= 90
    Grade$ = "A"
  Case 80 To 89
    Grade$ = "B"
  Case Else
    Grade$ = "F"
End Select
```

Figure 14. FOR, SELECT CASE and EXIT FOR

Data

Statement	Syntax	Description
DATA	`DATA item, item, ...`	Defines constants for READ. Items are numbers or strings; a string needs quotes only if it contains a comma. DATA may be anywhere in the program.
READ	`READ var, var, ...`	Assigns the next DATA items to the variables. Reading past the last item is error 4, Out of DATA.
RESTORE	`RESTORE [label|line]`	Resets the data pointer to the first DATA item, or to the first at or after the given line.

Figure 15. Data statements

Program control and environment

Statement	Syntax	Description
RANDOMIZE	`RANDOMIZE [seed]`	Seeds the random number generator. Without a seed the clock is used. The same seed gives the same series.
CALL	`CALL name[(args)]`	Calls a SUB. A SUB can also be called without CALL and without parentheses: name arg, arg.
DEF FN	`DEF FNname[(params)] = expr`	Defines a single-line function; see Chapter 5.
ERROR	`ERROR n`	Raises error number n as if it had occurred.
SLEEP	`SLEEP [seconds]`	Waits. Without a value, waits for a key (a line of input).

Statement	Syntax	Description

CLS, LOCATE, COLOR	`CLS`
	`LOCATE row, col`
	`COLOR fg [, bg]`
		Screen control. They act only under the -ansi option.

BEEP	`BEEP`	Sends the bell character.
SCREEN, WIDTH, VIEW, PALETTE, SHELL, COMMON, LSET, RSET, KILL		Accepted and ignored, so that existing programs load.

Figure 16. Other statements

Chapter 5. Procedures and Functions

SUB and FUNCTION

A SUB performs an action; a FUNCTION returns a value. Both may have parameters, local variables and recursion. A procedure may be defined before or after the statements that call it, and a DECLARE statement, accepted for compatibility with older programs, does nothing.

```
SUB name [(param [AS type], ...)]
  statements
END SUB

FUNCTION name [(param [AS type], ...)] [AS type]
  statements
END FUNCTION
```

Figure 17. Procedure definitions

The value of a function is set by assigning to its name, by assigning to the word FUNCTION, or by RETURN with a value (which also leaves the function at once). EXIT SUB and EXIT FUNCTION leave a procedure early.

```
Function Fib(n As Integer) As Integer
  If n < 2 Then Return n
  Return Fib(n - 1) + Fib(n - 2)
End Function

Sub Swap2(ByRef a As Integer, ByRef b As Integer)
  Dim t As Integer = a
  a = b
  b = t
End Sub

Dim x As Integer = 1, y As Integer = 2
Swap2 x, y
Print "x ="; x; " y ="; y
For i = 0 To 15: Print Fib(i);: Next

x = 2  y = 1
 0  1  1  2  3  5  8  13  21  34  55  89  144  233  377  610
```

Figure 18. A recursive function and a SUB with BYREF parameters

Parameters

Parameters are passed by reference unless BYVAL is given: the procedure then works on the caller's variable, and an assignment to the parameter changes it. An array is passed by writing its name with empty parentheses, a(), and the procedure sees the same array. A call with fewer arguments than parameters leaves the remaining parameters zero or empty. A string argument passed to a numeric parameter, or the reverse, is a Type mismatch.

Scope of variables

A variable used in a SUB or FUNCTION is local to it, and is created afresh at each call, unless it is declared DIM SHARED or CONST in the main program, or STATIC in the procedure. Variables of the main program are not visible in a procedure unless shared. Recursion saves and restores the locals of each activation, to a depth of 400 calls.

DEF FN

The older single-line function is defined with DEF FN and used like a function whose name begins with FN. The parameters are ordinary variables that are saved while the function is evaluated.

```
10 DEF FNSQ(X) = X * X
20 PRINT FNSQ(7)

 49
```

Figure 19. DEF FN

GOSUB

GOSUB transfers to a label or line number and RETURN comes back. The stack of return addresses is separate from the stack of procedure calls. Inside a SUB or FUNCTION, RETURN with no GOSUB pending returns from the procedure; this lets FreeBASIC programs use RETURN as they expect.

Names that duplicate built-in functions

A SUB or FUNCTION whose name is also that of a built-in function (MAX, say, or TRIM) replaces the built-in function for calls that have an argument list.

Chapter 6. Input and Output

PRINT

PRINT writes the values of its expressions. The separator after an expression controls the position of the next: a semicolon leaves the cursor where it is, and a comma moves it to the next print zone, which is 14 columns wide. A statement that does not end with either separator ends the line. The question mark is a short form of PRINT.

In MS style a number is written with a space where a plus sign would be and a space after it, so that PRINT 1; 2 shows " 1 2 ". In FB style the trailing space is not written. A negative number shows its minus sign in the place of the leading space. Strings are written as they are. TAB(n) moves to column n, and SPC(n) writes n spaces.

```
PRINT "a";"b";: PRINT "c"
PRINT 3;-3;
PRINT
FOR I=1 TO 3 : PRINT I; : NEXT : PRINT
PRINT "tab:"; TAB(8); "x", "y"

abc
 3 -3
 1  2  3
tab:   x      y
```

Figure 20. PRINT separators

A number whole in value is written without a decimal point, up to 15 digits; other numbers have up to 15 significant digits, with trailing zeros removed, and very large or small values use exponent form (1E+20).

PRINT #n, writes to file n opened for OUTPUT or APPEND, with the same rules.

PRINT USING

PRINT USING fmt$; list formats each item of the list by a field of the format string. Text in the format that is not part of a field is written as it is; if there are more items than fields, the format is used again. The figure below lists the fields.

Field	Meaning
#	a digit position
.	decimal point position
,	in the integer part, commas between thousands
+ at the start	always write the sign of the number
- or + at the end	sign after the number (a space for a positive number if -)
$$	a dollar sign written just before the first digit
**	fill unused leading positions with asterisks
^^^^	exponent form: 1.235E+04
\ \	a string field; its width is the two backslashes and the spaces between
!	the first character of a string
&	the whole string
_	the next character of the format is a literal

Figure 21. PRINT USING fields

If the number does not fit the field, a percent sign is written in front of it, and the number is written in full. Numbers are rounded, not truncated, to the decimal places of the field.

```
10 PRINT USING "Item: \     \ Qty: ### Price: $$###.## Total: #,###.##"; "Widget"; 12; 4.5;
12 * 4.5

Item: Widget  Qty:  12 Price:    $4.50 Total:    54.00
```

Figure 22. PRINT USING

INPUT and LINE INPUT

INPUT ["prompt" ;|,] var, var, ... reads a line from the keyboard and assigns its comma-separated fields to the variables. A semicolon after the prompt adds a question mark; a comma does not. If a field is not a valid number, the message ?Redo from start is written and the whole input is repeated. If the line has too few fields the interpreter asks for more with ??. A string field may be quoted to contain commas.

LINE INPUT ["prompt";] var$ reads a whole line, with its spaces and commas, into one string variable. Both statements accept #n, to read from a file instead of the keyboard. Reading when no more input exists is error 62, Input past end of file.

```
10 INPUT "Name"; N$
20 INPUT "Two numbers: ", A, B
30 PRINT "Hi "; N$; A + B

Name? Bob
Two numbers: 3, 4
Hi Bob 7
```

Figure 23. INPUT in a session

Files

Files are sequential text files, numbered from 1 to 255. FREEFILE returns the lowest number that is not in use.

```
OPEN name$ FOR INPUT|OUTPUT|APPEND AS [#]n
OPEN "I"|"O"|"A", [#]n, name$
CLOSE [[#]n [, [#]n ...]]
```

Figure 24. Opening and closing

OUTPUT creates the file or empties it, APPEND adds to the end, INPUT reads. CLOSE with no numbers closes all files, and they are closed when the program ends. Opening a file that does not exist for INPUT is error 53, File not found; opening a number that is already open is error 55. For a file open for input, EOF(n) is true (-1) when no more lines remain. The statements that use files are PRINT #, WRITE #, INPUT # and LINE INPUT #.

WRITE writes its items separated by commas, with strings in quotes and no spaces around numbers, in the form that INPUT reads back.

```
Open fname For Output As #1
For i = 1 To 5
  Print #1, "Line number"; i
Next
Close #1
Open fname For Input As #2
Do While Not Eof(2)
  Line Input #2, ln
  n += 1
  Print n; ": "; ln
Loop
Close #2

 1 : Line number 1
 2 : Line number 2
 3 : Line number 3
 4 : Line number 4
 5 : Line number 5
```

Figure 25. Writing and reading a file

Reading the keyboard directly

INPUT$(n) reads n characters, and INKEY$ reads one. Both wait for input, as the interpreter reads its input a character at a time from the standard input; INKEY$ does not return an empty string when no key is pressed.

Chapter 7. Built-in Functions

A function whose result is a string has a name that ends in a dollar sign in Microsoft BASIC (LEFT$); FreeBASIC allows the dollar sign to be omitted (Left). Both forms are accepted for every function in this chapter. A function that takes no arguments, such as TIMER or RND, needs no parentheses. Using the wrong type of argument is error 13, Type mismatch.

Numeric functions

Function	Result
ABS(x)	absolute value
SGN(x)	sign: -1, 0 or 1
INT(x)	largest whole number not greater than x: INT(-3.5) is -4
FIX(x)	x with the fraction removed: FIX(-3.5) is -3
FRAC(x)	the fraction of x
CINT(x), CLNG(x)	x rounded to a whole number (halves round to even)
CSNG(x), CDBL(x)	x as a floating-point number
CBOOL(x)	-1 if x is not zero, otherwise 0
SQR(x)	square root; error 5 if x is negative
SIN(x), COS(x), TAN(x)	trigonometric functions; x in radians
ATN(x), ATAN(x)	arc tangent
ATAN2(y, x)	arc tangent of y/x, in the right quadrant
ASIN(x), ACOS(x)	arc sine and cosine; error 5 outside -1 to 1
SINH(x), COSH(x), TANH(x)	hyperbolic functions
EXP(x)	e to the power x
LOG(x), LOG10(x), LOG2(x)	logarithms; error 5 if x is not positive
POW(x, y)	x to the power y
MAX(a, b), MIN(a, b)	the larger and the smaller of two numbers
RND[(x)]	random number from 0 up to but not including 1. RND(0) repeats the last value; RND with a negative value reseeds the generator.
TIMER	seconds from a fixed origin, with fractions
VAL(s$)	the number at the start of the string; &H, &O and &B constants are understood; 0 if there is none
VALINT(s$)	VAL with the fraction removed
ISNUMERIC(s$)	-1 if the whole string is a valid number, otherwise 0
IIF(cond, a, b)	a if cond is not zero, otherwise b; a and b may be strings

Figure 26. Numeric functions

String functions

Function	Result
LEN(s$)	length of the string
LEFT$(s$, n)	the first n characters
RIGHT$(s$, n)	the last n characters
MID$(s$, start [, n])	n characters from position start (the first character is 1); to the end if n is omitted
MID$(s$, start [, n]) = t$	as a statement, replaces part of s$ with the start of t$
INSTR([start,] s$, t$)	position of t$ in s$, or 0
INSTRREV(s$, t$ [, start])	position of the last t$ in s$, or 0
LTRIM$(s$), RTRIM$(s$), TRIM$(s$)	remove blanks from the left, the right or both
UCASE$(s$), LCASE$(s$)	convert to upper or lower case

Function	Result
STR$(x)	the number as text; in MS style a positive number has a leading space
CHR$(n)	the character with code n
ASC(s$)	the code of the first character
SPACE$(n)	n blanks
STRING$(n, c)	n copies of a character; c is a one-character string or a character code
HEX$(n), OCT$(n), BIN$(n)	the number in hexadecimal, octal and binary
TAB(n), SPC(n)	used in PRINT; see Chapter 6

Figure 27. String functions

System and file functions

Function	Result
DATE$	the date as mm-dd-yyyy
TIME$	the time as hh:mm:ss
COMMAND$[(n)]	the words that followed the program name on the command line; word n if given
ENVIRON$(name$)	the value of an environment variable
ERR, ERL	number and line of the last error; see Chapter 8
FREEFILE	lowest unused file number
EOF(n)	-1 if file n has no more data
INPUT$(n [, #f])	n characters from the keyboard or file f
INKEY$	one character from the keyboard
POS(0)	the current column of the cursor, from 1
LBOUND(a [, n]), UBOUND(a [, n])	lowest and highest subscript of dimension n of an array
FRE(x), CSRLIN	return fixed values for compatibility

Figure 28. System and file functions

Chapter 8. Errors and Error Handling

Runtime errors

An error ends the program with a message on the standard error stream unless the program has an error handler. The message gives the text of the error and the line. The tokenizer and the link pass can find errors before the program runs: an END IF without IF, a NEXT without FOR, a block that is not closed, or an EXIT outside any block of its kind. They are reported as "basic: ... at line n", and the program does not start.

Code	Message	Typical cause
1	NEXT without FOR	NEXT with no active FOR loop
2	Syntax error	a statement that the interpreter cannot parse
3	RETURN without GOSUB	RETURN in the main program with no GOSUB pending
4	Out of DATA	READ after the last DATA item
5	Illegal function call	argument out of range, such as SQR(-1) or LEFT$(a$, -1)
6	Overflow	EXP of a very large number
7	Out of memory	an array too large, or recursion more than 400 deep
8	Undefined line number	GOTO, GOSUB or RESTORE of a label that does not exist
9	Subscript out of range	array subscript outside the bounds
11	Division by zero	/, \ or MOD with a zero divisor
13	Type mismatch	string used where a number is needed, or the reverse
20	RESUME without error	RESUME when no error is being handled
52	Bad file name or number	file number outside 1 to 255
53	File not found	OPEN FOR INPUT of a file that does not exist
54	Bad file mode	reading a file opened for output, or the reverse
55	File already open	OPEN of a number that is in use
62	Input past end of file	INPUT or LINE INPUT at end of input
70	Permission denied	OPEN of a file that cannot be written

Figure 29. Error codes

ON ERROR

```
ON ERROR GOTO label|line    enable a handler
ON ERROR GOTO 0             disable it
ON ERROR RESUME NEXT        ignore errors; continue after the failing statement
RESUME                      run the failing statement again
RESUME NEXT                 continue after the failing statement
RESUME label|line           continue at a label
ERROR n                     raise error n
```

Figure 30. Error handling statements

When an error occurs in the main program and a handler is enabled, ERR holds the error number and ERL the line number (or the source line), and control passes to the handler. The handler ends with a RESUME. An error in a SUB or FUNCTION leaves the procedure and is handled in the main program, at the statement that made the call. An error inside a handler, before its RESUME, ends the program.

```
70 ON ERROR GOTO 500
80 PRINT 1 / 0
90 PRINT "after error"; ERR; ERL
500 PRINT "error"; ERR; "at"; ERL: RESUME NEXT

error 11 at 80
after error 11  80
```

Figure 31. An error handler

Chapter 9. How BASIC/CREXX Works

This chapter is for those who want to read or change the interpreter. The source file is organized in parts, in the order of the figure below.

Part	Content
Header	Option parsing, the shared state (the CREXX namespace exposes all arrays to all procedures), initialization, the keyword and function lists and the operator precedence table.
Tokenizer	Loads the program, splits each line into tokens, resolves names to variable slots, collects DATA items, recognizes compound keywords such as END IF and EXIT FOR.
Link pass	Matches block structures, finds the ranges of SUB and FUNCTION, builds the parameter and local-variable tables.
Evaluator	Expressions, arrays, argument lists, calls of procedures and DEF FN functions, runtime errors.
Functions	The built-in functions, string and number conversions, the random number generator.
Input/output	PRINT, PRINT USING, INPUT, DATA and READ, file statements.
Executor	The statement loop, assignment, and each control statement.
Main	Starting a program, token dump, interactive mode.

Figure 32. Parts of basic.crexx

Tokens

The program is held in parallel arrays of type, number, text, kind, flags and two link fields. Token types are number, string, name, keyword, operator, end of line, end of program and type name. A name token holds the index of its variable slot, so that no lookup by name is needed at run time. Variables live in arrays indexed by slot; array elements live in one pool of numbers and one pool of strings.

The link pass

The link fields of a keyword token hold the position of the token it is related to. An IF points to the next ELSEIF, ELSE or END IF; a FOR to the end of its NEXT; a DO to the end of its LOOP; a SELECT CASE to its first CASE. This makes the execution of a block statement a jump and not a search. Labels and line numbers are kept in a table, and the first use of a label stores its address in the label token.

Execution

The executor reads the token at the current position. A statement begins with a keyword or a name. FOR loops keep a stack of the variable, limit, step and return position. GOSUB has a stack of return positions. A call of a SUB or FUNCTION saves the local variables of the callee, binds the parameters, runs the body with a nested statement loop, copies BYREF parameters back and restores the locals.

Errors set a flag; each level returns at once and the statement loop of the main program tests it and decides whether a handler is active.

Performance

A loop of one million iterations with one assignment takes about 2.6 seconds on the system on which the interpreter was developed, that is roughly one million simple statements per second. A variable reference, a literal and a plain assignment have fast paths that avoid the general evaluator.

Building

basic.crexx is compiled by the crexx driver: crexx basic.crexx -noexec compiles it to basic.rxbin without running it. The compiler prints a few warnings about loop variables reused in sibling blocks; they are harmless.

Chapter 10. Restrictions and Known Problems

- **User-defined types.** TYPE ... END TYPE, UNION and ENUM are not supported. Programs that use them stop with a syntax error.
- **Random-access files.** OPEN FOR RANDOM or BINARY, PUT, GET, SEEK, FIELD, LOF and LOC are not supported.
- **Graphics and sound.** SCREEN is accepted and ignored. PSET, LINE, CIRCLE, PAINT, SOUND, PLAY and the like are not supported.
- **Preprocessor.** #include, #define and the other directives are ignored; a program cannot include another file.
- **Precision.** Numbers are 64-bit floating point; SINGLE is not rounded to 7 digits as in GW-BASIC, so PRINT 1/3 shows 15 digits. Integer types do not overflow; they are rounded only. Integer values beyond 2 to the power 63 are clamped.
- **Number formatting.** Small numbers print in plain decimal, not in the E form of the Microsoft interpreters (0.000015, not 1.5E-05).
- **Keyboard.** INKEY$ and SLEEP with no argument wait for input; there is no way to test for a key without waiting. Line editing is that of the terminal.
- **Screen.** CLS, LOCATE and COLOR do nothing unless -ansi is given. The position of the cursor is tracked only for TAB and POS.
- **Arrays.** At most three dimensions. ERASE and REDIM do not free the storage used by the old array. Local arrays of procedures called many times use new storage each call.
- **Limits.** Procedure calls may be nested 400 deep; GOSUB 20000 deep. The size of an array is limited to 30 million elements.
- **Speed.** The interpreter is itself interpreted by the CREXX virtual machine; programs run about 100 to 1000 times more slowly than under a compiled BASIC.
- **The C version.** It differs from the CREXX version only for text outside the 7-bit character set (it works on bytes, CHR$ above 255 gives the low byte) and for hexadecimal literals of more than 15 digits.
- **Line ends.** A carriage return at the end of a program line is ignored, so files from DOS can be used unchanged. A program file must be plain text.

Appendix A. Command Summary

Command line

```
./basic [-ms|-qb|-fb] [-ansi] [-dump] program.bas [arguments]
./basic                       interactive mode
crexx basic.crexx -args [options] program.bas [arguments]
crexx basic.crexx -nocompile -args [options] program.bas [arguments]
```

Figure 33. Synopsis

Environment

Variable	Meaning
CREXX	Full path of the crexx driver used by the launcher, if it is not on the PATH.

Figure 34. Environment variables

Return codes

Code	Meaning
0	the program ended normally, or END was executed
1	a runtime error was not handled, or the program had link errors, or could not be opened
2	the launcher could not compile the interpreter

Figure 35. Return codes

Appendix B. Contents of the Repository

Path	Content
basic.crexx	the interpreter, one CREXX Level B source file
basic	the launcher script: compiles the interpreter on first use, then runs it
README.md	a short description of the project
docs/BASIC-CREXX-Guide.docx	this manual
examples/	sample programs; see Appendix D
c/	the C89 port: src/ (the parts), basic_all.c (the amalgamated single source), Makefile, tools/, tests/run_tests.sh, deck/basic_deck.c (71-column card deck)
bench/	benchmark programs for both versions and bench/run_bench.sh
tests/run_tests.sh	the regression test driver
tests/*.bas, *.out, *.in, *.flags	test programs with their expected output, standard input and options
.gitignore	excludes compiled files (basic.rxbin and the like)

Figure 36. Files of the repository

Appendix C. Differences between the Dialects

BASIC/CREXX accepts programs for both dialects at once and does not need to be told which one a program is written in; only the layout of printed numbers differs (the -ms and -fb options). The figure below lists where the three languages differ and what BASIC/CREXX does.

Feature	Microsoft BASIC	FreeBASIC	BASIC/CREXX
PRINT of a positive number	space, number, space	space, number	selected by -ms and -fb
Array lower bound	0 (OPTION BASE 1)	0	0, OPTION BASE 1, or lo TO hi
Integer type	16 bit, overflows	32 or 64 bit	rounded 64-bit float; no overflow
Floating type	single, 7 digits	double, 15 digits	double, 15 digits
SUB, FUNCTION	QuickBASIC only	yes	yes
Line numbers	required in GW-BASIC	optional	optional
Variables in a SUB	local	local	local
RETURN value in a FUNCTION	not available	RETURN expr	supported
String concatenation	+	+ and &	+ and &
Compound assignment	no	yes	yes
EXIT, CONTINUE	EXIT only	both	both
TYPE	QuickBASIC only	yes	no
Graphics, sound	yes	yes	no

Figure 37. Dialect differences

Appendix D. Sample Programs

The directory examples holds the programs in the figure below. Each one is also a regression test.

File	Dialect	Shows
hello.bas	MS, numbered	PRINT, FOR, SQR, print zones
primes.bas	FB	CONST, arrays with bounds, nested loops, compound assignment
subs.bas	FB	FUNCTION, SUB, BYREF, recursion, SELECT CASE
strings.bas	MS, numbered	string functions, loops over characters
using.bas	MS, numbered	PRINT USING fields
fileio.bas	FB	OPEN, PRINT #, LINE INPUT #, EOF
hanoi.bas	MS, numbered	SUB with DECLARE, DIM SHARED, recursion
mandel.bas	FB	floating point, DIM AS DOUBLE, EXIT FOR, an ASCII Mandelbrot set
guess.bas	MS, numbered	INPUT, RND, RANDOMIZE TIMER, GOTO

Figure 38. Sample programs